OOPS ka matlab kya hai?

OOPS = Object-Oriented Programming System

OOPS kya hai ?

OOPS ek programming style hai jisme hum real-world cheezon ko "objects" ke roop mein represent karte hain.

Isme data (properties) aur functions (methods) ko objects ke andar organize karte hain, taaki code samajhna, likhna aur maintain karna easy ho jaye.

(

OOPS is a programming style where we model real-world things as "objects".
Each object contains data (called properties) and functions (called methods) that operate on the data.
This helps organize code better, makes it easier to understand, reuse, and maintain.

)

Pillar	Purpose	Example Concept
Encapsulation	Hides internal state	Private fields, closures
Abstraction	Exposes only needed parts	Public methods
Inheritance	Reuses logic in child classes	`class Dog extends Animal`
Polymorphism	Same interface, different behavior	`animal.makeSound()`

📄 Class and Object –

✅ 1. What is a Class?

- **Definition:** Class ek blueprint ya template hoti hai jisme hum object banane ke liye data (properties) aur functions (methods) define karte hain.

What does a Class contain ?

A class mainly contains:

1. **Properties (Attributes):** These are the data or characteristics of the object. Example: color, model, speed for a car.

2. **Methods (Functions):** These are actions or behaviors the object can perform. Example: start(), stop(), accelerate() for a car.

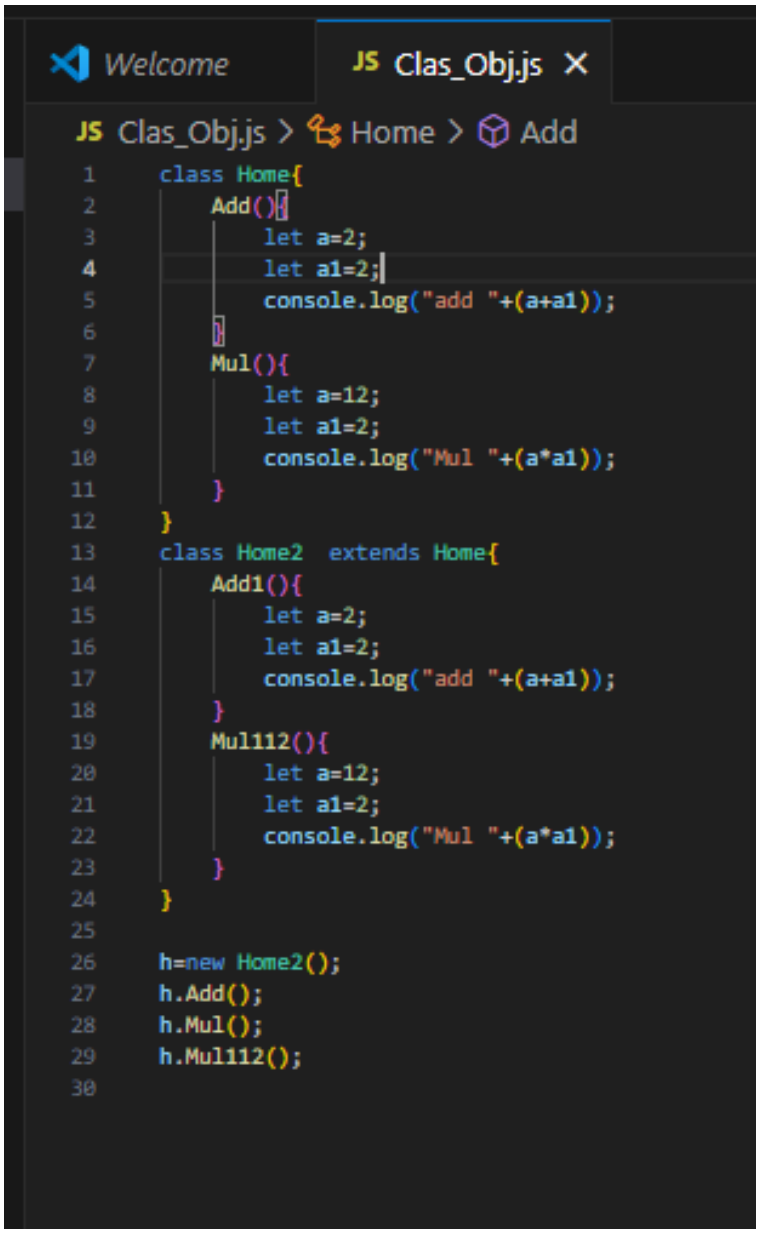
```
class ClassName {
   method1() {
      // code
   }
   method2() {
      // code
   }
}
```

🔷 Types of Inheritance in OOP (Object-Oriented Programming)

Traditionally, there are **5 types** of inheritance:

Type	Meaning
1. **Single**	One child class inherits from one parent
2. **Multi-level**	One class inherits from another, which inherits from another
3. **Hierarchical**	Multiple child classes inherit from one parent
4. **Multiple**	One class inherits from **multiple** parents
5. **Hybrid**	Combination of multiple types

1. Single→

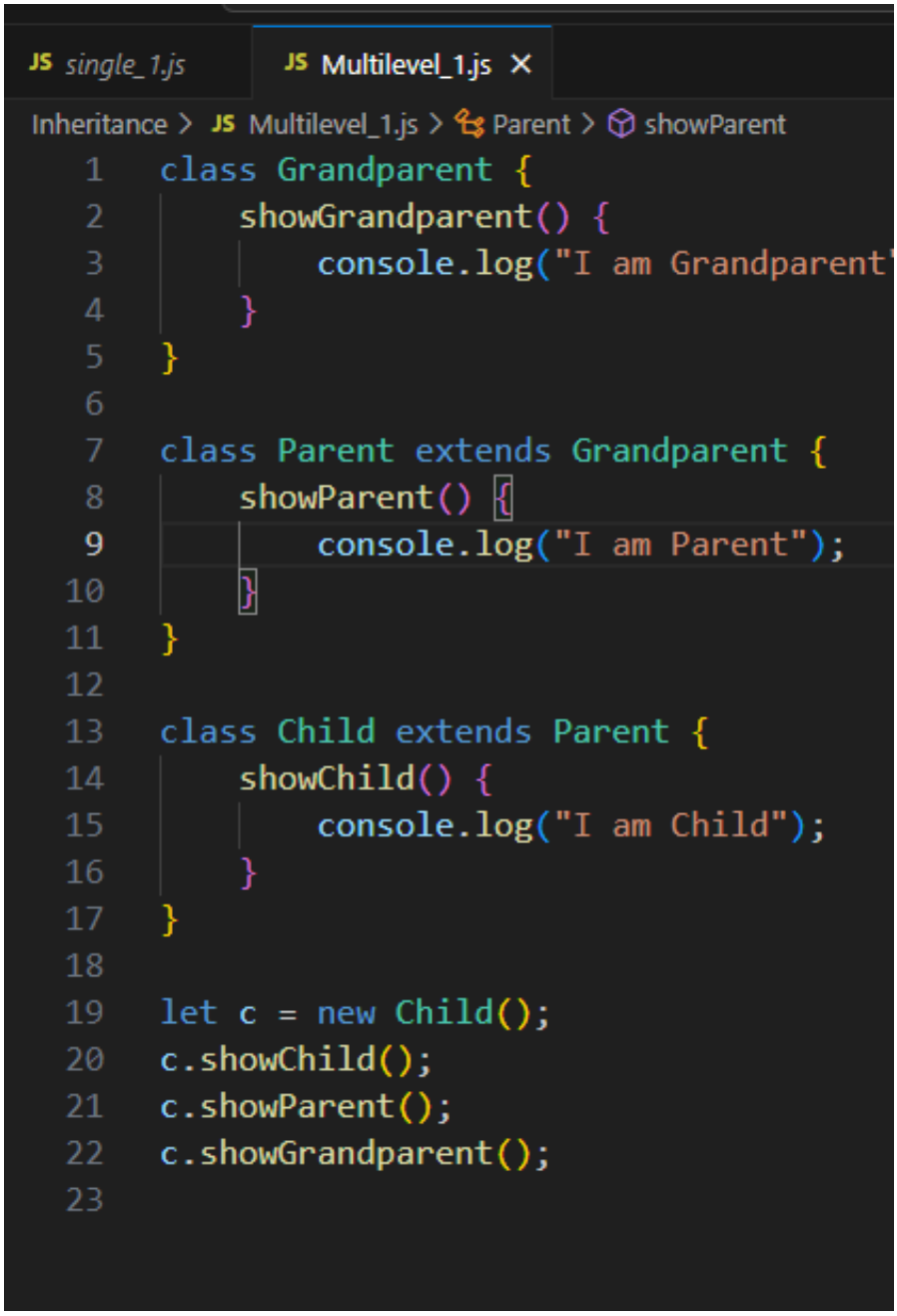


Code →click me

2 What is Multilevel Inheritance?

Multilevel Inheritance me ek class **dusri class se inherit** karti hai, aur wo class **teesri class**

se inherit karti hai.
Jaise ek **chain** banti hai.

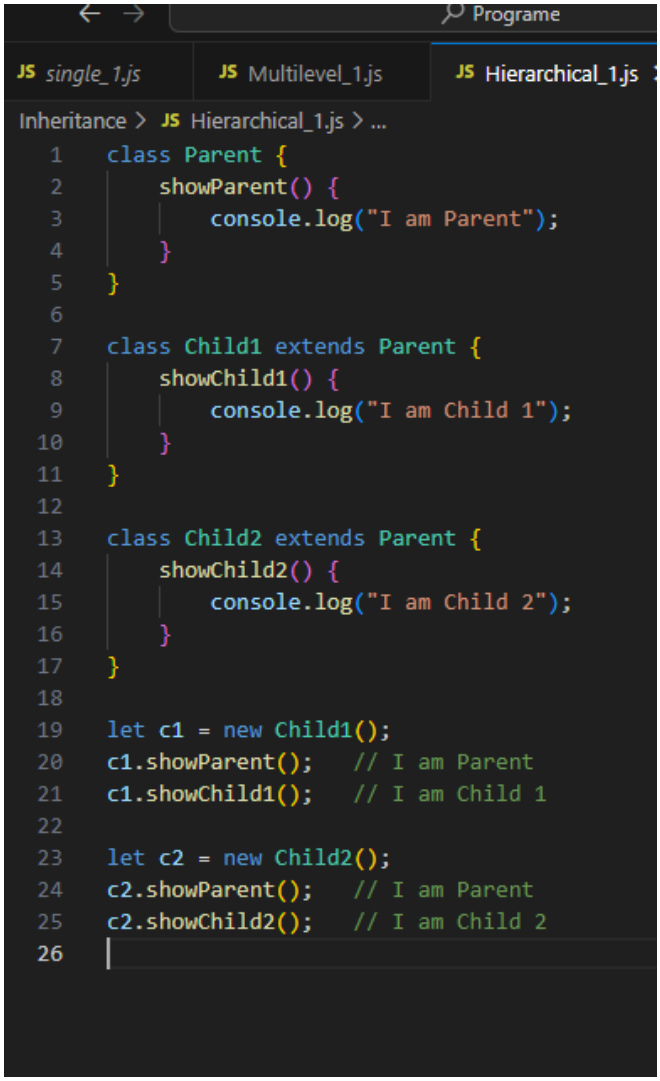


Code →

Clicke me

3. Hierarchical Inheritance

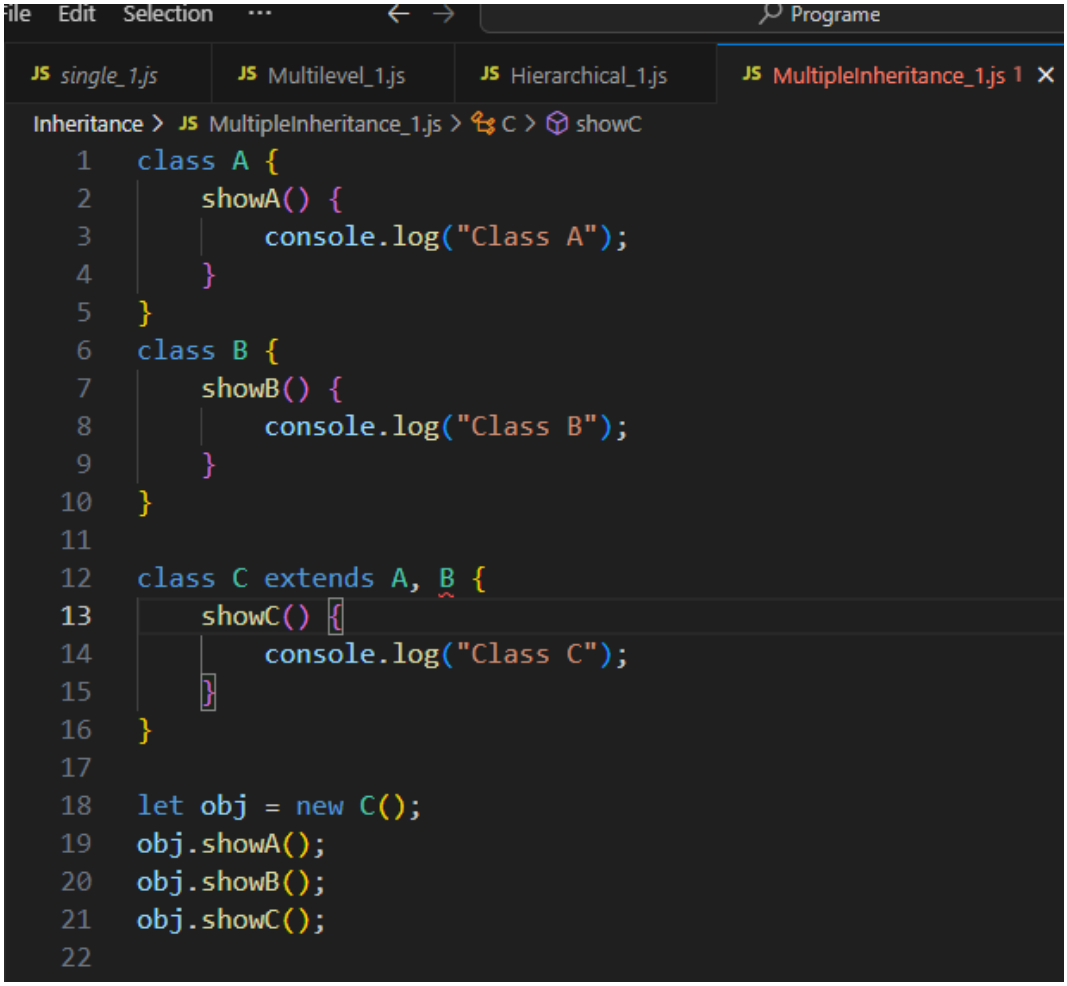
One parent class, but **multiple child classes** inherit from it.



Code -> [Click me]

Multiple Inheritance ➔

```
// ❌ ERROR: JavaScript doesn't allow extending multiple classes
```



📌 Why Error?

- JavaScript only allows one `extends` keyword at a time.
- You **cannot inherit from two classes directly** like `class C extends A, B`.

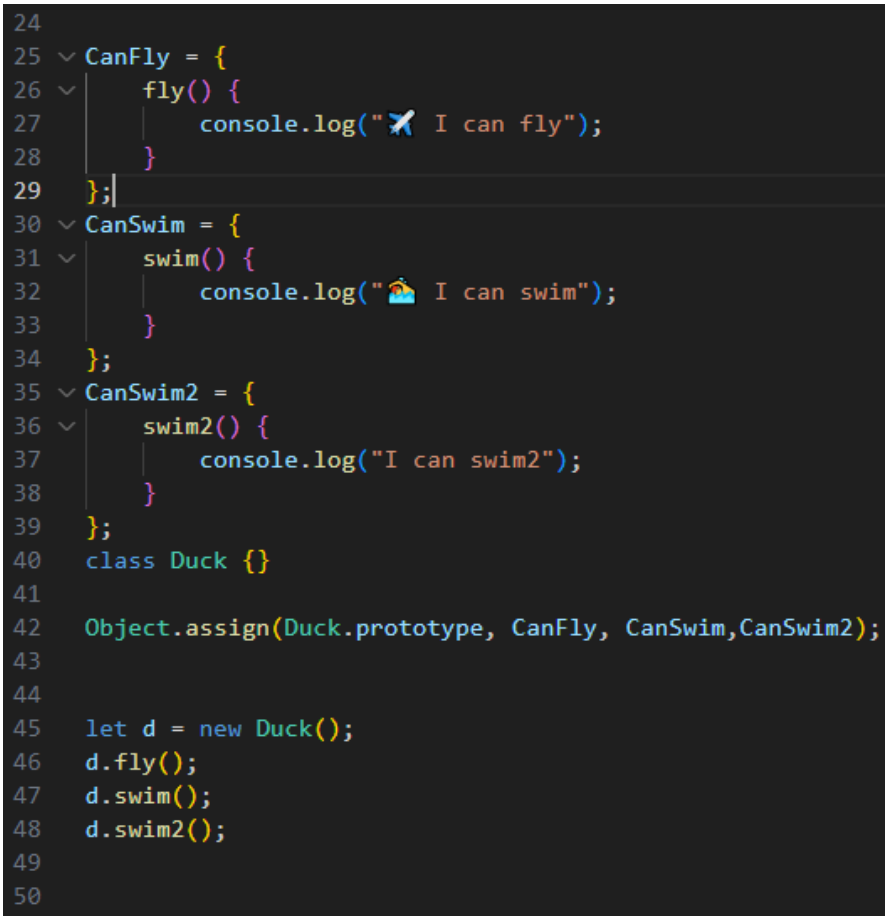
MIXIN METHOD ➔ :

What is a Mixin?

Mixin ek aisa technique ya object hota hai jisme hum kuch reusable methods (functions) rakhte hain, aur unhe alag-alag classes me **copy karke** use karte hain.

Why use Mixin?

- JavaScript me **multiple inheritance** nahi hoti.
- Lekin agar tumhe ek class me do ya zyada classes ke behavior chahiye, to tum **Mixin** use karte ho.
- Isse tum ek class me **bahut saare features** add kar sakte ho bina inheritance ke.



```
24
25  CanFly = {
26      fly() {
27          console.log("✈️ I can fly");
28      }
29  };
30  CanSwim = {
31      swim() {
32          console.log("🏊 I can swim");
33      }
34  };
35  CanSwim2 = {
36      swim2() {
37          console.log("I can swim2");
38      }
39  };
40  class Duck {}
41
42  Object.assign(Duck.prototype, CanFly, CanSwim,CanSwim2);
43
44
45  let d = new Duck();
46  d.fly();
47  d.swim();
48  d.swim2();
49
50
```

Code →click me

Prototypal Inheritance

Prototype का use हम तब करते हैं:

- जब हम चाहते हैं कि एक object या function के जरिए कई object बनें और सबमें एक जैसा behavior (methods) हो।
- और वो shared behavior memory efficient हो।

ये वाला कोड **ES6 Classes और Inheritance** वाला तरीका है |

```
class Parent {
  greet() {
    console.log("Hello from Parent");
  }
}

class Child extends Parent {
 greet() {
    console.log("Hello from Pare121212nt");
  }
}

const child = new Child();
child.greet();
```

Super method ➔

📌 `super` क्या है?

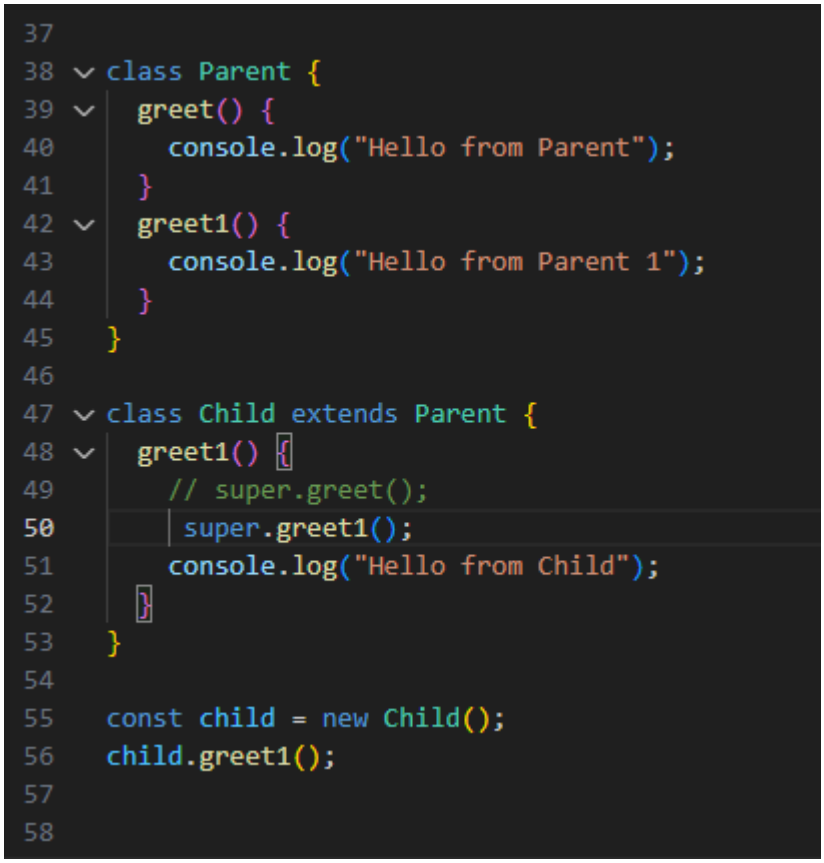
`super` keyword का इस्तेमाल JavaScript में **parent class के constructor या methods को call करने के लिए** किया जाता है।

```
class Parent {
  greet() {
    console.log("Hello from Parent");
  }
}

class Child extends Parent {
  greet() {
    super.greet();           // Parent का greet() कॉल कर रहा हूँ
    console.log("Hello from Child");
  }
}

const child = new Child();
child.greet();
```

o/p → ?



```
37
38  class Parent {
39    greet() {
40      console.log("Hello from Parent");
41    }
42    greet1() {
43      console.log("Hello from Parent 1");
44    }
45  }
46
47  class Child extends Parent {
48    greet1() {
49      // super.greet();
50      super.greet1();
51      console.log("Hello from Child");
52    }
53  }
54
55  const child = new Child();
56  child.greet1();
57
58
```

code →click me

- ## Q: Constructor ka use kya hai?

Constructor ek special function hota hai class ke andar, jo **object banate hi automatically chal jaata hai**.

📌 Use:

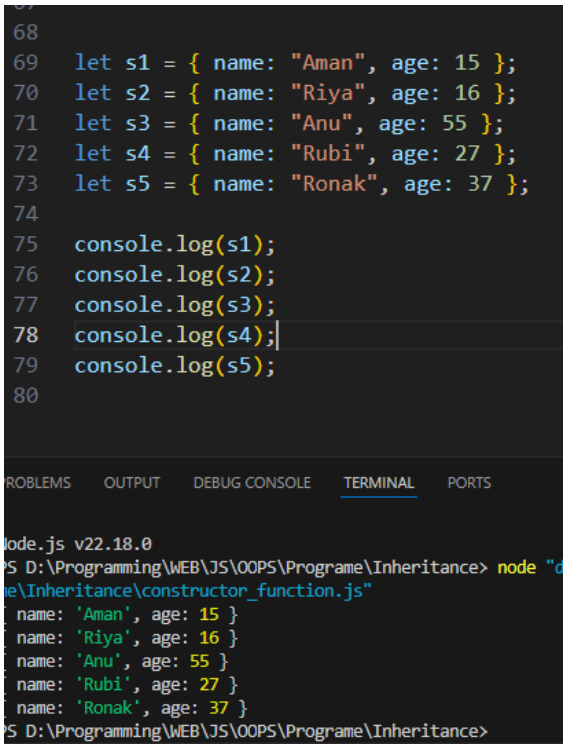
- Jab aap chahte ho ki **har object ke andar same structure** ho (same properties ho).
- Aapko **multiple similar objects** banana ho alag-alag data ke saath.

Q: Constructor kyu use karte hain? (Kya zarurat hai?)

👉 Jab aap:

1. Har object me **same properties** repeat kar rahe ho (e.g., `name`, `age`)
2. Multiple objects bana rahe ho (10, 50, 100...)
3. Future me code maintain karna easy rakhna chahte ho

W/o constructor →



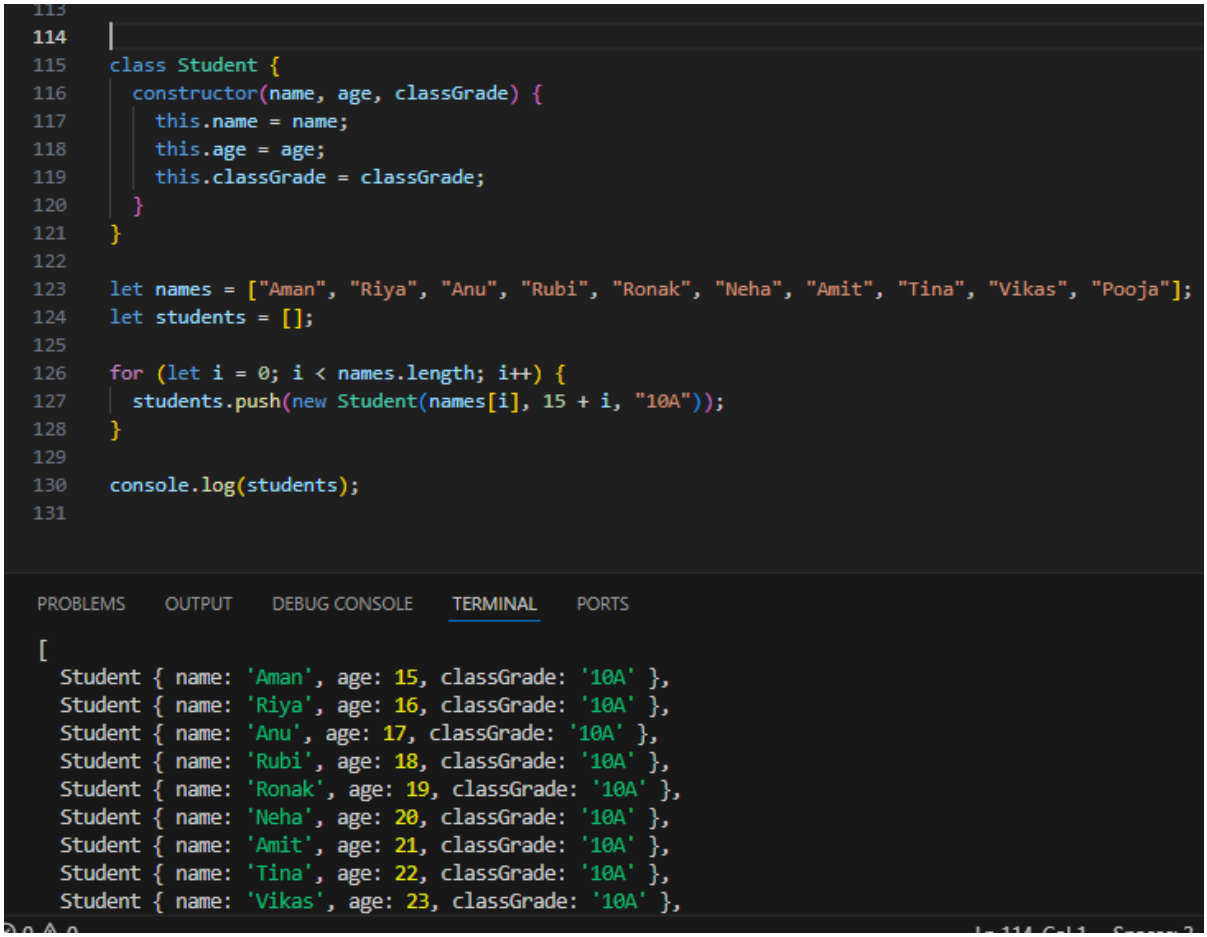
Code → click me

```
80
81  let s1 = { name: "Aman", age: 15, fatherName: "Ramesh", motherName: "Sita" };
82  let s2 = { name: "Riya", age: 16, fatherName: "Sanjay", motherName: "Geeta" };
83  let s3 = { name: "Anu", age: 55, fatherName: "Mahesh", motherName: "Lata" };
84  let s4 = { name: "Rubi", age: 27, fatherName: "Raj", motherName: "Meena" };
85  let s5 = { name: "Ronak", age: 37, fatherName: "Amit", motherName: "Sunita" };
86
87  console.log(s1, s2, s3, s4, s5);
88
89
90
91
```

With Constructor ➔

```
94
95  class Student {
96    constructor(name, age, fatherName, motherName) {
97      this.name = name;
98      this.age = age;
99      this.fatherName = fatherName;
100     this.motherName = motherName;
101   }
102 }
103
104 let s1 = new Student("Aman", 15, "Ramesh", "Sita");
105 let s2 = new Student("Riya", 16, "Sanjay", "Geeta");
106 let s3 = new Student("Anu", 55, "Mahesh", "Lata");
107 let s4 = new Student("Rubi", 27, "Raj", "Meena");
108 let s5 = new Student("Ronak", 37, "Amit", "Sunita");
109
110 console.log(s1, s2, s3, s4, s5);
111
```

◆ **Constructor ke saath (loop se bana do 10 student):**



Click me → code

✅ Polymorphism in JavaScript

Polymorphism ka matlab hota hai:

"Ek hi method naam, lekin alag objects usko alag tareeke se implement karte hain."

🔁 **Poly = many**, **Morph = forms**

Yani: **same function, different behavior depending on the object.**

Type	Naam	Kab hota hai?
1️⃣	**Compile-time Polymorphism**	Method Overloading (Mostly in Java)
2️⃣	**Runtime Polymorphism**	Method Overriding (Java & JavaScript dono me)

1. Compile-time Polymorphism (Static Polymorphism)

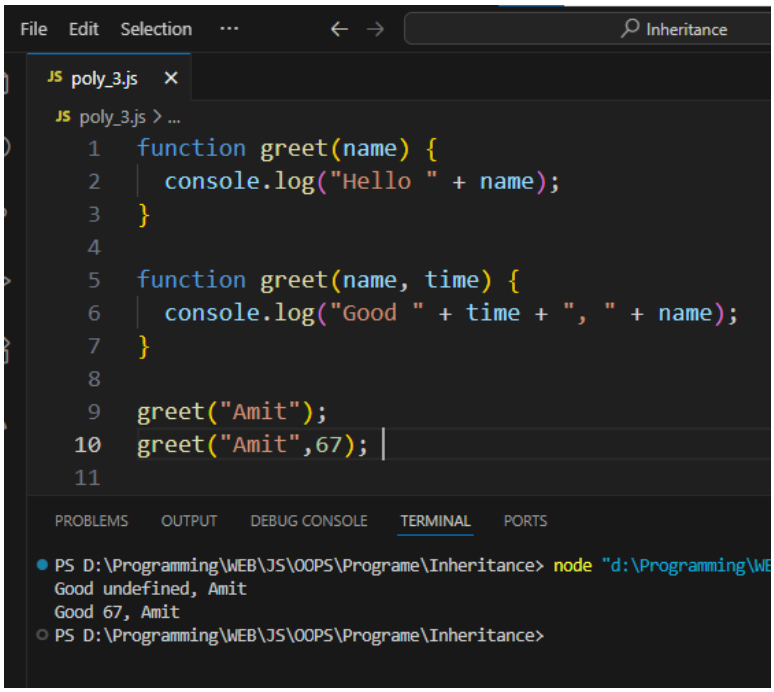
Method Overloading ➔

◆ Kya hota hai?

- **Same method name, different parameters.**
-
- Compiler decide karta hai kaun sa method run karega.
-
- Java me hota hai, lekin **JavaScript me nahi**.

⚠ JavaScript me kyu nahi?

- JS me function overloading officially supported nahi hai.
- Agar same naam se do function likhe, to **last wala *overwrite* kar deta hai**.



Click me → Code

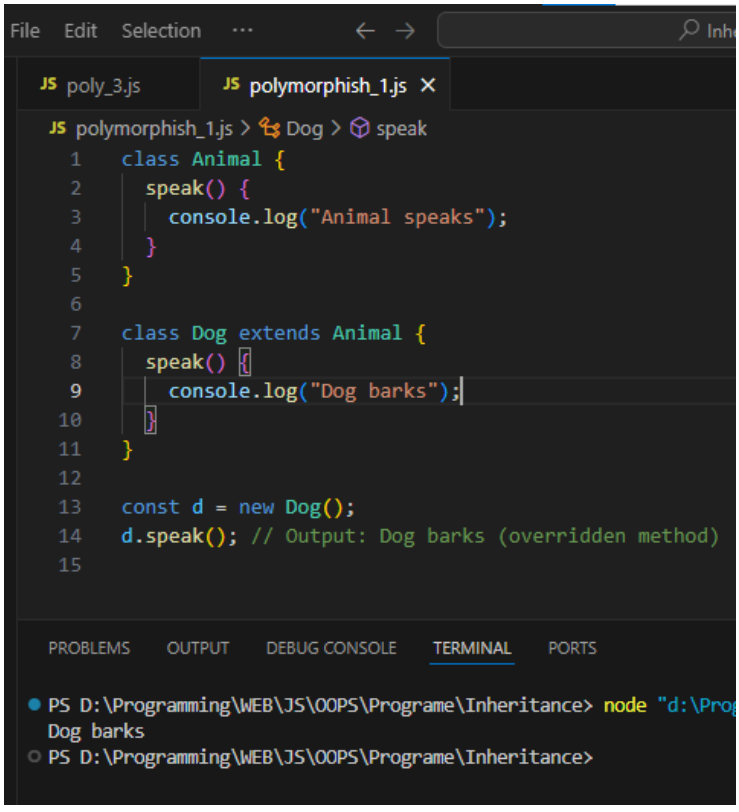
✅ 2. Runtime Polymorphism (Dynamic Polymorphism)

◆ Kya hota hai?

- **Same method name**, parent-child relationship me.
- **Child class** apne hisaab se method ka behavior change karti hai.
- **JavaScript me yahi hota hai** via method overriding.

🔁 Override ka Matlab Kya Hota Hai?

"Override" ka matlab hota hai **parent class ke method ko child class me dobara likhna, lekin alag behavior ke sath**.



Code → click me

🔐 Encapsulation Kya Hota Hai ?

Encapsulation ek Object-Oriented Programming (OOP) ka basic principle hai, jiska matlab hota hai:

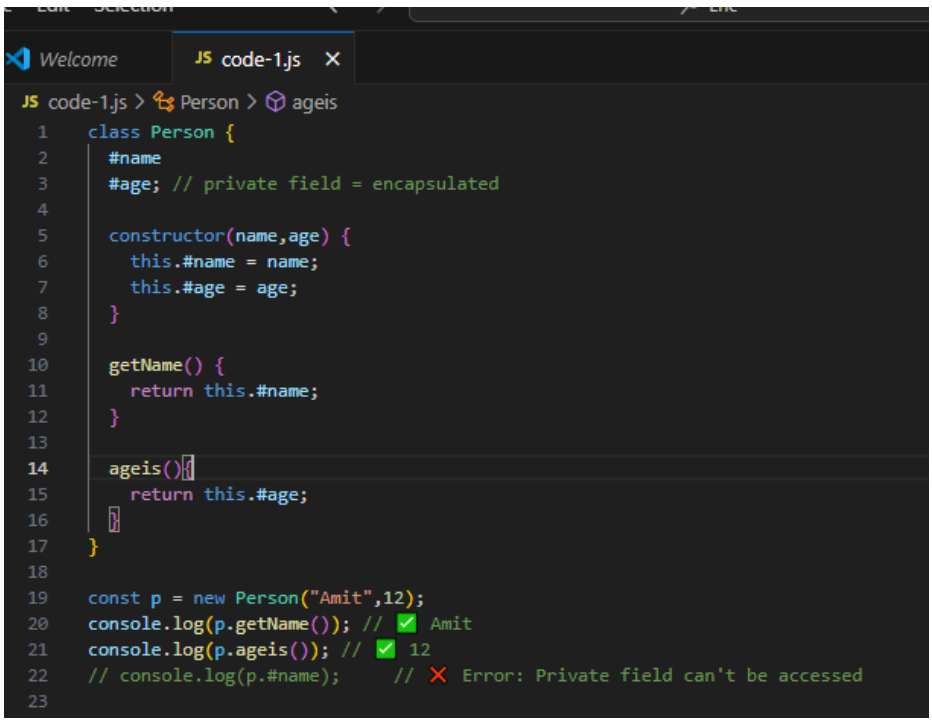
"Data (properties) aur us data par kaam karne wale methods (functions) ko ek single unit ke andar band kar dena, aur direct access ko rokna."

🎁 Simple Shabdon Mein:

Encapsulation ka matlab hai:

- Apne **data ko chhupana (hide karna)**.
- Sirf **methods ke through data tak access dena**.
- Isse data safe rehta hai, galti se koi change nahi kar sakta.

- • **Data protection** – koi bhi direct data ko modify na kare.
- • **Control** – kaun kya access kar sakta hai, ye control mein rehta hai.
- • **Security** – sensitive data chhupa rehta hai.
- • **Simplicity** – code clean aur easy to use hota hai.



Click me→ Code